

Amdahl Reality Gap - Written Analysis

Zeba Academy / Parallel Computing - Practicum Worksheet

Student and machine details

Student Full Name	Tulegen Yerassyl
Student ID Number	230103342
Institutional Email / Git URL	https://github.com/Aqyn97/314_110926 (folder: PRACTICUM WORKSHEET)
Practicum Date & Time (Slot)	02-N , 08-P
Laptop Model & OS	MacBook Air, macOS
CPU Model & Generation	Apple M2 (Apple silicon, 2nd generation)
Physical Cores (P/E breakdown)	8 (4 performance + 4 efficiency)
Logical SMT Threads (omp_get_max_threads)	8 (no SMT on Apple M-series; physical = logical)
Target Workload Size N	13,342,000 (= 10,000,000 + 3342 x 1,000)
L1 / L2 Cache Line Size	128 bytes (sysctl hw.cachelinesize)

Measured data (from results.csv)

Sequential baseline: Run 1 = 2.094530 s (cold, discarded), Run 2 = 2.051099 s, Run 3 = 2.042641 s, so $T_{seq} = 2.046870$ s. Checksum 111680116, max stopping time 688, 10,766,131 numbers above 100 steps (identical in every configuration).

Table 1 - scaling ($p = 0.926$ from $k = 2$)

Threads k	Run 1 (cold)	Run 2 (s)	Run 3 (s)	Avg T_k (s)	S_{emp}	S_{theo}	Delta
1	2.040146	2.033173	2.039292	2.036233	1.005	1.000	-0.005
2	1.097485	1.099174	1.099244	1.099209	1.862	1.862	0.000
4	0.573441	0.572933	0.572775	0.572854	3.573	3.273	-0.300
8 (max)	0.388842	0.387723	0.393139	0.390431	5.243	5.269	+0.027

Table 2 - Experiment A, false sharing (8 threads)

Variant	Threads	Time (s)	Throughput (iter/s)	Penalty ratio
1: naive hit_count[tid]++ (false sharing)	8	0.406572	32,815,876	1.034
2: padded struct (128 B per counter)	8	0.393244	33,928,088	1.000

Table 3 - Experiment B, loop scheduling (8 threads)

Clause	Chunk	Time (s)	vs static	Scheduling behavior
static	N/8 (~1.67M)	0.381788	baseline	One contiguous block per thread
static, 1000	1,000	0.386097	+1.1%	Round-robin blocks of 1,000
dynamic, 100	100	0.385529	+1.0%	~133,420 chunks from a shared queue
dynamic, 10000	10,000	0.374839	-1.8%	~1,335 chunks from a shared queue
guided	decaying	0.370115	-3.1%	Large chunks first, shrinking toward the end

Implementation notes (differences from the worksheet)

- Variant 1 declares the naive counter array as volatile int, so the compiler cannot keep the counter in a register and every increment really reaches the shared cache line. The worksheet's version is a plain int array.
- Variant 2 pads each counter to 128 bytes instead of the worksheet's 64, because this M2 reports a 128-byte cache line (sysctl hw.cachelinesize). The padding size is set by -DCACHE_LINE=128 at compile time.
- The maximum thread count is $k = 8$. The M2 has 8 cores and no SMT, so $k = 16$ was not run and the last row of Table 1 is $k = 8$.
- The configuration $k = 8$, static was measured twice: 0.3904 s in Table 1 and 0.3818 s in Experiment B. The gap of about 2% is run-to-run noise on this machine.

Q1: Root cause of false sharing

Variant 1 (naive `hit_count[tid]++`) took 0.4066 s against 0.3932 s for Variant 2 (padded counters) at 8 threads: 32.8 M vs 33.9 M iter/s, a penalty ratio of 1.034 (3.4% slower). The gap is small, only slightly above my run-to-run noise of about 2-3%, so I report it as a measured but modest effect.

Mechanism: the eight 4-byte counters are adjacent, so together they occupy 32 bytes inside one cache line. On this M2 the line is 128 bytes (`hw.cachelinesize = 128`). Coherence is tracked per line, not per variable. Each thread only writes its own counter, but to do so its core must hold the line in the Modified (exclusive) state. Under a MESI/MOESI-style protocol this sends invalidations to every other core holding a copy, and those copies become Invalid. The next thread that writes must then fetch the line back from the previous owner. The line ping-pongs between cores although no data is logically shared, which creates invalidation and cache-to-cache transfer traffic and stalls the stores. Variant 2 gives each counter its own line, which removes the sharing. I padded to 128 bytes rather than the worksheet's 64, because a 64-byte pad would still let two threads' counters share one 128-byte line on this CPU.

Why the penalty is small here: each number needs a Collatz loop of typically over a hundred steps (80.7% of the numbers took more than 100), so the counter update is a small part of the work and most of the coherence stalls are hidden behind computation. A loop that updates the shared line more often per unit of compute should show a much larger penalty, but I did not test that.

Q2: SMT saturation and physical core ceiling

This question does not apply directly to my machine. The Apple M2 has no SMT, so physical cores and logical threads are both 8. I could not increase k beyond the physical core count and did not run $k = 16$.

What my data shows is scaling up to the full core count: speedup 1.86 ($k = 2$), 3.57 ($k = 4$), 5.24 ($k = 8$), which is efficiency of 93%, 89% and 65%. Going from 4 to 8 threads gave only 1.47x. This is not SMT saturation. Likely contributors are the 4 efficiency cores being slower than the 4 performance cores, lower clocks under sustained all-core load in a fanless MacBook Air, and growing fork/join and reduction overhead. I did not isolate which of these dominates.

In general, SMT threads on one physical core share the execution units (ALUs, load/store units), the branch predictor, the L1/L2 caches and TLB, and the front end; only the architectural register state is duplicated. A compute-bound kernel like Collatz therefore gains little from SMT, so speedup flattens beyond the physical core count. My M2 cannot demonstrate this.

Q3: Empirical vs theoretical Amdahl discrepancy

Derived parallel fraction: $p = 2 \times (1 - 1/S_{\text{emp}}(2)) = 2 \times (1 - 1/1.862) = 0.926$, using $T_{\text{seq}} = 2.0469$ s and $T_2 = 1.0992$ s. The fitted sequential fraction is 7.4%, so Amdahl's ceiling is $1/(1-p) = 13.5$.

On my machine the curves stayed close: $\Delta = -0.005$ ($k = 1$), 0.000 ($k = 2$, by construction), -0.300 ($k = 4$), $+0.027$ ($k = 8$). At $k = 4$ the measured speedup (3.573) was above the model (3.273). A true Amdahl bound with a fixed p cannot be exceeded, which shows that p fitted from a single point is not a constant and absorbs noise and other effects. The close match at $k = 8$ should not be over-interpreted, since one fitted parameter can match a few points. Efficiency itself still dropped from 89% at $k = 4$ to 65% at $k = 8$.

Two physical factors Amdahl's equation omits: (1) Hardware heterogeneity and clock behavior: 4 performance and 4 efficiency cores run at different speeds, and clocks can drop under all-core load, so k workers are not k identical units. (2) Parallel runtime overhead and load imbalance: thread fork/join, barrier and reduction costs, plus uneven iteration cost (26 needs 10 steps, 27 needs 111), which make

the effective p depend on k . Memory bandwidth and cache-coherence traffic are also omitted, but this compute-bound kernel barely stresses them (Experiment A).

Q4: Scheduling trade-off

At 8 threads: static 0.3818 s, static,1000 0.3861 s, dynamic,100 0.3855 s, dynamic,10000 0.3748 s, guided 0.3701 s. The total spread is about 4%. That is the same size as my noise: the identical configuration ($k = 8$, static) measured 0.3904 s in Table 1 and 0.3818 s here (2.2% apart), and runs 2 and 3 of a single configuration differ by up to about 3.6%. I therefore cannot claim a firm ranking; in this measurement guided and dynamic,10000 were fastest.

dynamic(100) makes about 133,420 visits to the shared work queue against about 1,335 for dynamic(10000), roughly 100 times more. It was 2.9% slower (0.3855 vs 0.3748 s), which points in the direction of queue overhead but is within noise. Static was as good as dynamic because each thread receives about 1.67 million consecutive numbers, and the uneven Collatz costs average out over blocks that large, leaving little imbalance for dynamic scheduling to fix.

Crossover: I did not observe a chunk size where lock-queue contention clearly outweighed the balancing benefit. Even the smallest tested chunk (100) was not measurably worse than static. Locating the crossover would need smaller chunks (for example 1 to 10), which I did not test. On this machine, guided or dynamic with a large chunk is the sensible choice.